

How to Deploy a Full Microservices E-Commerce Application (11 Services) on AWS EKS



Yaswanth Reddy Arumulla

Follow

8 min read · Sep 22, 2025



20



1

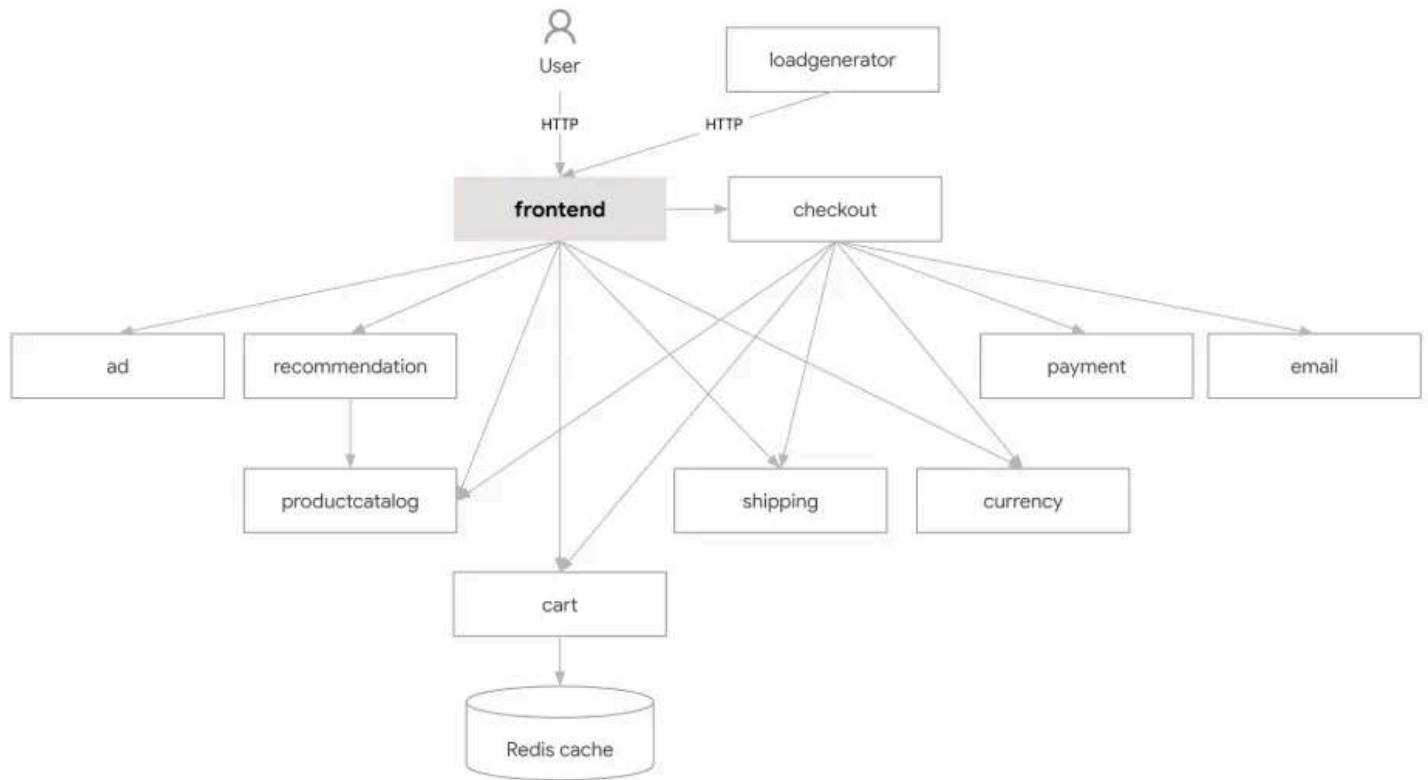


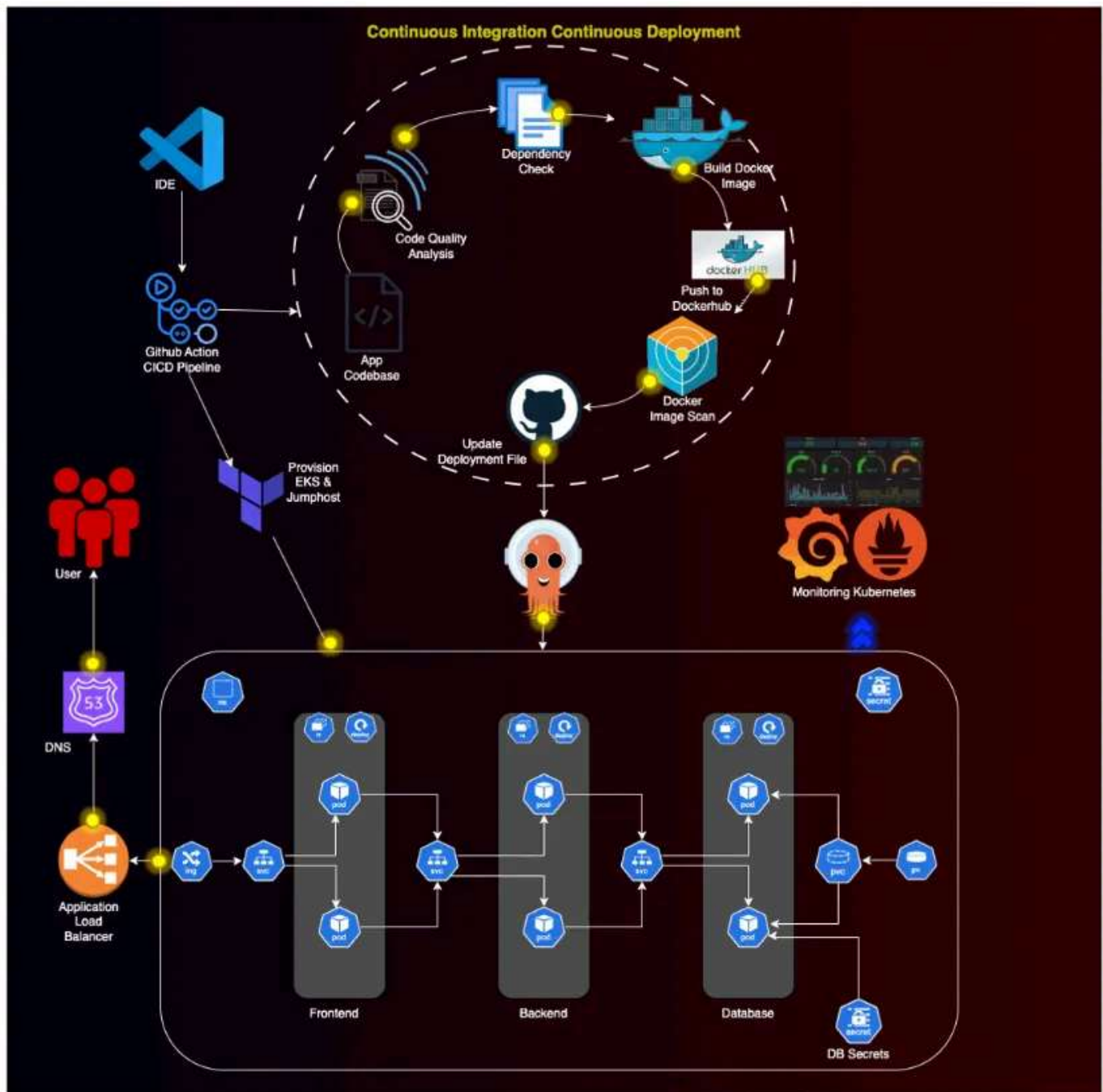
11 Microservice CI/CD Pipeline DevOps Project | E-Commerce A

Yash Academy



Tonton di





Deploying a **production-ready microservices e-commerce platform** on **Amazon EKS (Elastic Kubernetes Service)** requires orchestrating multiple services, CI/CD pipelines, observability tools, and infrastructure automation.

In this blog, we'll walk through the **complete setup** from cloning the repository to configuring Jenkins pipelines, deploying with ArgoCD, and

exposing the frontend with Route 53 + HTTPS.

Step 1: Clone the GitHub Repository

Open VS Code → Terminal → Run:

```
git clone https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-proje
```

Step 2: Configure AWS Keys

```
aws configure
```

Provide:

- Access Key ID
- Secret Access Key
- Region (e.g., `us-east-1`)
- Output format: `json`

Step 3: Navigate into the Project

```
ls  
cd Microservices-E-Commerce-eks-project
```

```
ls
```

Step 4: Create S3 Buckets for Terraform State

Terraform stores state files in S3 for collaboration.

```
cd s3-buckets/  
terraform init  
terraform plan  
terraform apply -auto-approve
```

Step 5: Create Network Infrastructure

Navigate to Terraform EC2 setup:

```
cd ../terraform_main_ec2  
terraform init  
terraform plan  
terraform apply -auto-approve
```

Sample output:

```
Apply complete! Resources: 24 added, 0 changed, 0 destroyed.  
jumphost_public_ip = "18.208.229.108"  
region = "us-east-1"
```

Check Terraform state:

```
terraform state list
```

Step 6: Connect to EC2 and Access Jenkins

From AWS Console → EC2 → Connect → Switch to root:

```
sudo -i
```

Verify installed DevOps tools:

```
git --version  
java -version  
jenkins --version  
terraform -version  
mvn -v  
kubectl version --client --short  
eksctl version  
helm version --short  
docker --version  
trivy --version
```

Get Jenkins admin password:

```
cat /var/lib/jenkins/secrets/initialAdminPassword
```

- example output :

```
0c39f23132004d508132ae3e0a7c70e4
```

Step 7: Jenkins Setup in Browser

Open:

```
http://<EC2 Public IP>:8080
```

- Paste admin password
- Install suggested plugins
- Create first user (example: yaswanth)
- Click through: Save and Continue → Save and Finish → Start using Jenkins

Step 9: Install Jenkins Plugins

1. Go to Jenkins Dashboard → Manage Jenkins → Plugins.
2. Click the Available tab.
3. Search and install the following:

- ☒ Pipeline: stage view

4. when installation is complete:

- Restart jenkins when installation is complete and no job are running

Step 10: Create a Jenkins Pipeline Job (Create EKS Cluster)

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: eks-terraform

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : eks-terraform/eks-jenkinsfile
- Apply
- Save

6. click Build with Parameters

- ACTION :
- Select Terraform action : apply
- Build

To verify your EKS cluster, connect to your EC2 jump host server and run:

```
aws eks --region us-east-1 update-kubeconfig --name project-eks
kubectl get nodes
```

Step 11: Create a Jenkins Pipeline Job (Create Elastic Container Registry (ecr))

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: ecr-terraform

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : ecr-terraform/ecr-jenkinsfile
- Apply
- Save

6. click Build with Parameters

- ACTION :
- Select Terraform action : apply
- Build

To verify your EKS cluster, connect to your EC2 jumphost server and run:

```
aws ecr describe-repositories --region us-east-1
```

Services:

- emailservice
- checkoutservice
- recommendationservice
- frontend
- paymentservice
- productcatalogservice
- cartservice
- loadgenerator
- currencyservice
- shippingservice
- adservice

Step 12: Create a Jenkins Pipeline Job for Build and Push Docker Images to ECR

Step 12.1: Add GitHub PAT to Jenkins Credentials

1. Navigate to Jenkins Dashboard → Manage Jenkins → Credentials → (global) → Global credentials (unrestricted).
2. Click “Add Credentials”.
3. In the form:
 - Kind: Secret text
 - Secret: ghp_HKMTPOddKYE2LfDLGuytsimfedgdssxnnl5d1f73zh
 - ID: my-git-pattoken
 - Description: git credentials
4. Click “OK” to save.

Step 12.2: Jenkins Pipeline Setup: Build and Push and update Docker Images to ECR

Step 12.2.1: Jenkins Pipeline Setup: emailservice

1. Go to Jenkins Dashboard
2. Click New Item
3. Name it: emailservice

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/emailservice
- Apply
- Save

6. click Build

Step 12.2.2: Jenkins Pipeline Setup: checkoutservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: checkoutservice

4. Select: Pipeline

5. Click OK

- Pipeline:

- **Definition** : Pipeline script from SCM
- **SCM** : Git
- **Repositories** : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- **Branches to build** : */master
- **Script Path** : jenkinsfiles/checkoutservice
- **Apply**
- **Save**

6. click Build

Step 12.2.3: Jenkins Pipeline Setup: recommendationservice

1. Go to Jenkins Dashboard
2. Click New Item
3. Name it: recommendationservice
4. Select: Pipeline
5. Click OK

- **Pipeline:**
- **Definition** : Pipeline script from SCM
- **SCM** : Git
- **Repositories** : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>

- Branches to build : */master
- Script Path : jenkinsfiles/recommendationservice
- Apply
- Save

6. click Build

Step 12.2.4: Jenkins Pipeline Setup: frontend

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: frontend

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/frontend
- Apply
- Save

6. click Build

Step 12.2.5: Jenkins Pipeline Setup: paymentservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: paymentservice

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/paymentservice
- Apply
- Save

6. click Build

Step 12.2.6: Jenkins Pipeline Setup: productcatalogservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: `productcatalogservice`

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : `*/master`
- Script Path : `jenkinsfiles/productcatalogservice`
- Apply
- Save

6. click Build

Step 12.2.7: Jenkins Pipeline Setup: cartservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: `cartservice`

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/cartservice
- Apply
- Save

6. click Build

Step 12.2.8: Jenkins Pipeline Setup: loadgenerator

1. Go to Jenkins Dashboard
2. Click New Item
3. Name it: loadgenerator
4. Select: Pipeline
5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>

- Branches to build : */master
- Script Path : jenkinsfiles/loadgenerator
- Apply
- Save

6. click Build

Step 12.2.9: Jenkins Pipeline Setup: currencyservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: currencyservice

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/currencyservice
- Apply
- Save

6. click Build

Step 12.2.10: Jenkins Pipeline Setup: shippingservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: shippingservice

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/shippingservice
- Apply
- Save

10. click Build

Step 12.2.11: Jenkins Pipeline Setup: adservice

1. Go to Jenkins Dashboard

2. Click New Item

3. Name it: adservice

4. Select: Pipeline

5. Click OK

- Pipeline:
- Definition : Pipeline script from SCM
- SCM : Git
- Repositories : <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Branches to build : */master
- Script Path : jenkinsfiles/adservice
- Apply
- Save

6. click Build



Step 13: Install ArgoCD in Jumphost EC2

13.1: Create Namespace for ArgoCD

```
kubectl create namespace argocd
```

13.2: Install ArgoCD in the Created Namespace

```
kubectl apply -n argocd \  
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install
```

13.3: Verify the Installation

Ensure all pods are in Running state.

```
kubectl get pods -n argocd
```

13.4: Validate the Cluster

Check your nodes and create a test pod if necessary:

```
kubectl get nodes
```

13.5: List All ArgoCD Resources

```
kubectl get all -n argocd
```

Sample output:

NAME	READY	STATUS	RESTART
pod/argocd-application-controller-0	1/1	Running	0
pod/argocd-applicationset-controller-787bfd9669-4mxq6	1/1	Running	0
pod/argocd-dex-server-bb76f899c-slg7k	1/1	Running	0
pod/argocd-notifications-controller-5557f7bb5b-84cjr	1/1	Running	0
pod/argocd-redis-b5d6bf5f5-482qq	1/1	Running	0
pod/argocd-repo-server-56998dcf9c-c75wk	1/1	Running	0
pod/argocd-server-5985b6cf6f-zzgx8	1/1	Running	0

13.6: Expose ArgoCD Server Using LoadBalancer

13.6.1: Edit the ArgoCD Server Service

```
kubectl edit svc argocd-server -n argocd
```

13.6.2: Change the Service Type

Find this line:

```
type: ClusterIP
```

Change it to:

```
type: LoadBalancer
```

Save and exit (:wq for vi).

13.6.3: Get the External Load Balancer DNS

```
kubectl get svc argocd-server -n argocd
```

Sample output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
argocd-server	LoadBalancer	172.20.1.100	a1b2c3d4e5f6.elb.amazonaws.com

13.6.4: Access the ArgoCD UI

Use the DNS:

```
https://<EXTERNAL-IP>.amazonaws.com
```

13.7: Get the Initial ArgoCD Admin Password

```
kubectl get secret argocd-initial-admin-secret -n argocd \
-o jsonpath="{.data.password}" | base64 -d && echo
```

Login Details:

- Username: admin
- Password: (The output of the above command)

Step 15: Deploying with ArgoCD and Configuring Route 53 (Step-by-Step)

Step 15.1: Create Namespace in EKS (from Jumphost EC2)

Run these commands on your jumphost EC2 server:

```
kubectl create namespace dev
kubectl get namespaces
```

Step 15.2: Create New Applicatio with ArgoCD

1. Open the ArgoCD UI in your browser.

2. Click + NEW APP.

3. Fill in the following:

- Application Name: `project`
- Project Name: `default`
- Sync Policy: `Automatic`
- Repository URL: <https://github.com/arumullayaswanth/Microservices-E-Commerce-eks-project.git>
- Revision: `HEAD`
- Path: `kubernetes-files`
- Cluster URL: <https://kubernetes.default.svc>
- Namespace: `dev`

4. Click Create.

Step 17: Create a Jenkins Pipeline Job for Backend and frontend & Route 53 Setup

Enable HTTPS for aluru.site with AWS Classic Load Balancer (CLB)

This guide explains how to configure HTTPS for your domain `aluru.site` using AWS Classic Load Balancer (CLB), Route 53, and AWS Certificate Manager (ACM).

Prerequisites

- A working application (e.g., on EC2 or Kubernetes).

- A registered domain: `aluru.site`
- Domain is managed in Route 53 as a Public Hosted Zone.

1. Go to AWS Route 53

2. Create a Hosted Zone:

- Domain: `aluru.site`
- Type: Public Hosted Zone

3. Update Hostinger Nameservers:

- Paste the 4 NS records from Route 53 into Hostinger:
- `ns-865.awsdns-84.net`
- `ns-1995.awsdns-97.co.uk`
- `ns-1418.awsdns-59.org`
- `ns-265.awsdns-73.com`

Your Classic Load Balancer is running and serving HTTP on port 80 or 8080.

Step 1: Request a Public Certificate in ACM

1. Go to AWS Certificate Manager (ACM).
2. Click Request Certificate.
3. Choose Request a Public Certificate.
4. Enter domain:

- `aluru.site`
- `www.aluru.site` (optional)

5. Choose DNS validation.

Get Yaswanth Reddy Arumulla's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

6. Click Request.

7. After request:

- Click Create DNS record in Route 53.
- ACM will create the `_acme-challenge` CNAME record.

8. Wait a few minutes until status becomes Issued.

Step 2: Add HTTPS Listener to CLB

1. Go to EC2 Console > Load Balancers.
2. Select your Classic Load Balancer.
3. Go to Listeners tab.
4. Click Add Listener (or edit existing 443):
 - Protocol: HTTPS

- Load Balancer Port: 443
- Instance Protocol: HTTP (or HTTPS if applicable)
- Instance Port: 80 (or 8080 if your app runs there)
- SSL Certificate: Choose the one for `aluru.site`
- Security Policy: Select `ELBSecurityPolicy-2021-06`

5. Click Save.

Step 3: Update Security Group Rules

Go to your EC2 or Load Balancer Security Group:

- Add Inbound Rule:
- Type: HTTPS
- Protocol: TCP
- Port: 443
- Source: `0.0.0.0/0`

Ensure existing rules allow HTTP (port 80) or your backend port.

Step 4: Configure DNS in Route 53

1. In ArgoCD UI, open your `project` application.
2. Click on frontend and copy the hostname (e.g.,
`acfb06fba08834577a50e43724d328e3-1568967602.us-east-1.elb.amazonaws.com`).
3. Go to Route 53 > Hosted Zones.

4. Select `aluru.site`.

5. Click Create Record:

- Record name: leave blank (for root domain)
- Record type: A — Routes traffic to an IPv4 address and AWS resource
- Alias: Yes
- Alias target: Choose Application and Classic Load Balancer
- Region: US East (N. Virginia)
- Alias target value: Paste the frontend load balancer DNS (from step 2)

6. Click Create Record.

Step 5: Test Your Setup

Using Browser

Visit:

```
https://aluru.site
```

You should see your application load securely over HTTPS.

Using curl

```
curl -v https://aluru.site
```

Expect HTTP 200 OK or the actual page content.

Troubleshooting

HTTPS times out?

- Check port 443 is open in Security Group.
- Make sure your app is reachable from the CLB.
- ACM certificate must be in Issued status.

HTTP works but HTTPS doesn't?

- Listener or certificate may not be configured properly.
- Check the load balancer health check passes.

Kubernetes Monitoring for Everyone: Step-by-Step with Prometheus & Grafana

<https://medium.com/@yaswanth.arumulla/kubernetes-monitoring-for-everyone-step-by-step-with-prometheus-grafana-b8582f0cf808>

DevOps

Ci Cd Pipeline

Docker

Kubernetes

Jenkins



Written by Yaswanth Reddy Arumulla

Follow

97 followers · 1 following

DevOps Engineer | Automating infrastructure | CI/CD enthusiast | Kubernetes, Docker, and Cloud-native

